

SECRETARÍA DE EDUCACIÓN PÚBLICA DE PUEBLA

DIRECCIÓN DE BACHILLERATOS ESTATALES Y PREPARATORIA ABIERTA (DBEPA)

Francisco Z. Mena

Clave C.C.T.: 21ECT0017T | Subsistema: BT

**LIBRO DE TEXTO Y CUADERNO DE TRABAJO ACTIVO: COMPENDIO
SEMESTRAL**

Formación Integral Basada en Misiones Comunitarias y Retos Tecnológicos

UAC: Programación (2° Semestre)

PROYECTO COMUNITARIO ESCOLAR (PAEC):

Desarrollo de software y automatización digital comunitaria para comercios y talleres locales de Francisco Z. Mena, mitigando la brecha digital y optimizando el inventario comunitario.

Docente Titular: Blizzard2 GF

Estudiante: _____

Grupo: _____ **Turno:** _____ **Ciclo Escolar: 2026-2027**

ÍNDICE DE MISIONES FORMATIVAS

Dosificación de misiones pedagógicas sesión por sesión vinculadas al plan de clase:

Misión	Título de la Misión	Sesiones Cubiertas	Páginas Est.
Bloque	%% BLOQUE I: BLOQUE 1 %%	Bloque I	50 págs.
Misión 1	Fundamentación e Intuición — Encuadre, normas de seguridad y diagnóstico de saberes previos	Sesiones: 1, 2, 3	6 págs.
Misión 2	Laboratorio y Taller Experimental — Análisis de especificaciones técnicas y procedimientos (Etapa 1)	Sesiones: 4, 5, 6, 7, 8, 9	8 págs.
Misión 3	Construcción del Artefacto Real — Análisis de especificaciones técnicas y procedimientos (Etapa 7)	Sesiones: 10, 11, 12, 13, 14, 15	6 págs.
Misión 4	Demostración Integral y Evaluación Formativa — Control de calidad, pruebas de funcionamiento y coevaluación	Sesiones: 16, 17, 18	5 págs.

MISIÓN 1: FUNDAMENTACIÓN E INTUICIÓN — ENCUADRE, NORMAS DE SEGURIDAD Y DIAGNÓSTICO

Sesiones asignadas: 1, 2, 3 | Enfoque: Encuadre, normas de seguridad y diagnóstico de saberes previos · Identificación de requerimientos técnicos, h

1. Enganche y Desafío Situado en la Comunidad

En el corazón de la región norte del estado de Puebla, específicamente en el municipio de Francisco Z. Mena, la vida comunitaria vibra al ritmo del comercio local, los talleres artesanales y las pequeñas empresas familiares que sustentan la economía de la zona. A pesar de la riqueza cultural y la tenacidad de sus habitantes, los tenderos, mecánicos y artesanos locales enfrentan un obstáculo invisible pero implacable: la brecha digital y la gestión manual de sus inventarios. Don Filemón, un comerciante de abarrotes con más de treinta años en la comunidad de Metlatoyuca, lleva el control de sus existencias en libretas desgastadas por el tiempo y la humedad. Cada tarde, tras cerrar las cortinas metálicas de su negocio, pasa horas buscando entre columnas de números por qué el frijol, el azúcar o el aceite no cuadran al final de la semana. Los errores humanos, el extravío de notas y la falta de respaldos físicos ponen en riesgo la estabilidad de su patrimonio y limitan las oportunidades de crecimiento para su familia. Este escenario no es un caso aislado; se repite en decenas de talleres mecánicos, fruterías y carpinterías de Francisco Z. Mena. Es aquí donde la programación y el pensamiento computacional dejan de ser conceptos abstractos contenidos en libros de texto para convertirse en herramientas de emancipación social y transformación tecnológica. Imagina que tú y tu equipo de compañeros de Educación Media Superior son convocados para diseñar una solución digital comunitaria: un sistema ligero, eficiente y adaptado a las condiciones locales que permita a Don Filemón y a otros comerciantes registrar entradas y salidas de mercancía con precisión milimétrica, utilizando equipos de cómputo básicos o incluso teléfonos celulares reciclados. Para lograr esta hazaña, antes de escribir una sola línea de código, debemos adentrarnos en los cimientos de la lógica, comprender cómo piensa una máquina y dominar las normas de seguridad y buenas prácticas que rigen a la industria del software a nivel internacional. Esta misión marca tu primer paso como arquitecto de software comunitario en Puebla.

Pregunta Detonadora: ¿Cómo podemos transformar un proceso manual y caótico de inventario en un sistema lógico, automatizado y seguro que resuelva un problema real en los comercios de Francisco Z. Mena?

2. Concepto Cero: Analogía Intuitiva y Fundamento

Analogía Física Cotidiana: Para entender qué es la programación, las variables, la memoria y la lógica computacional sin enredarnos en tecnicismos intimidantes, pensemos en una cocina tradicional poblana durante los preparativos para una gran festividad, como la elaboración de mole poblano para el día de la comunidad. Imagina que la cocina es la memoria RAM de una computadora y que la cocinera principal es el procesador (CPU). Sobre la mesa de trabajo hay decenas de ingredientes: chiles, ajonjolí, chocolate, especias y caldo. Si la cocinera dejara todos los ingredientes amontonados sin orden en una sola cubeta, encontrar el chocolate o calcular la cantidad exacta de chiles tomaría horas, y el resultado sería un caos total. Para evitar este desastre, la cocinera utiliza recipientes etiquetados. Toma un frasco de vidrio transparente y le pega una etiqueta con la palabra 'azúcar'. Dentro coloca exclusivamente azúcar. Toma otro recipiente y le pone 'caldo_pollo'. Ahora bien, ¿qué pasa si la cocinera necesita sumar ingredientes o realizar una receta compleja? Sigue una serie de pasos estrictos, una receta de cocina escrita en un recetario. Esa receta indica un orden riguroso: paso 1, tostar los chiles; paso 2, freír las semillas; paso 3, licuar todo con el caldo. Si la cocinera se salta el paso 2 y pasa directamente al paso 3 sin haber tostado los chiles, la receta fracasa. En la programación informática ocurre exactamente lo mismo. Una variable es simplemente ese frasco rotulado (la cajita en la memoria) que guarda un valor específico (un número, un texto, un resultado). El código fuente es nuestra receta de cocina: una secuencia ordenada, lógica e inquebrantable de instrucciones paso a paso que la computadora (nuestra eficiente pero literal ayudante de cocina) ejecuta sin cuestionar. Si nuestras instrucciones son claras, ordenadas y seguras, obtendremos un platillo perfecto; si hay ambigüedades, la computadora se detendrá o arrojará un error. Esta analogía física de la cocina y los recipientes rotulados es la piedra angular sobre la cual construiremos todo nuestro conocimiento técnico en esta unidad de aprendizaje.

La programación de computadoras no es magia negra ni una habilidad reservada para mentes geniales desconectadas de la realidad; es, en esencia, el arte y la ciencia de estructurar el pensamiento lógico para resolver problemas mediante instrucciones comprensibles para un sistema electrónico. En el contexto de la Educación Media Superior bajo el estándar industrial y tecnológico, debemos concebir la programación como un proceso de ingeniería riguroso. Todo sistema de software se sustenta en tres pilares fundamentales: la entrada de datos (lo que el usuario proporciona, como el nombre de un producto en la tienda de Don Filemón), el procesamiento (la manipulación matemática o lógica de esos datos, como calcular el costo total de la venta aplicando un impuesto o un descuento) y la salida de datos (el resultado mostrado al usuario, ya sea en pantalla, en una impresora de tickets o guardado en un archivo permanente).

Para profundizar en el concepto de 'Variable' desde una perspectiva industrial, debemos entender que la computadora cuenta con millones de casillas de memoria física llamadas bits y bytes. Gestionar directamente esas direcciones numéricas de memoria sería una tarea titánica y propensa a errores catastróficos. Por ello, los lenguajes de programación utilizan abstracciones llamadas variables. Una variable es un identificador simbólico (un nombre que nosotros elegimos) asociado a un espacio en la memoria RAM que puede almacenar un valor que cambiará o variará a lo largo de la ejecución del programa. Por ejemplo, en un sistema de inventario para comercios de Francisco Z. Mena, podríamos tener una variable llamada 'existencias_maíz'. Al iniciar el día, su valor es 100. Si un cliente compra 15 kilos, el programa toma el valor actual de la variable, le resta 15 y almacena el nuevo resultado (85) en el mismo espacio de memoria.

Es fundamental comprender los tipos de datos básicos que estas variables pueden albergar. En la industria del software, un dato no es solo un

número; tiene una naturaleza específica que determina cuánta memoria ocupa y qué operaciones se pueden realizar con él. Entre los tipos de datos primitivos más importantes encontramos:

1. Enteros (int): Números sin parte decimal, positivos o negativos, ideales para contar unidades de productos, piezas en stock o número de clientes (ej. 45, -3, 0).
2. Flujantes o Decimales (float / double): Números que contienen una parte fraccionaria separada por un punto, esenciales para manejar precios, pesos exactos de mercancía o porcentajes de IVA (ej. 15.50, 3.1416, 99.99).
3. Cadenas de Texto (string): Secuencias de caracteres alfanuméricos encerrados entre comillas, utilizados para nombres de productos, direcciones, descripciones y mensajes en pantalla (ej. 'Aceite Comestible 1 Litro', 'Metlatoyuca').
4. Booleanos (bool): Valores de lógica binaria que sólo pueden adoptar dos estados: Verdadero (True) o Falso (False), fundamentales para evaluar condiciones como ¿Hay suficiente inventario? (True si stock > 0, False si stock == 0).

Normas de seguridad y buenas prácticas industriales:

En el desarrollo de software profesional, la seguridad y la ética no son opcionales. Desde esta primera misión, debemos adoptar normas estrictas de la industria (como las convenciones de nomenclatura PEP 8 en Python o estándares ISO/IEC):

- Naming Conventions (Convenciones de nombres): Las variables deben tener nombres descriptivos en minúsculas separados por guiones bajos (snake_case), evitando nombres ambiguos como 'x', 'a' o 'temp'. En lugar de usar 'p', utilizaremos 'precio_producto'.
- Comentarios y Documentación: Todo código profesional debe estar documentado para que otros ingenieros o la propia comunidad puedan mantenerlo. Un código sin comentarios es una caja negra peligrosa.
- Gestión de Errores y Seguridad Física: En un entorno industrial o comercial, un programa nunca debe fallar de manera imprevista ante un error del usuario (por ejemplo, que el usuario ingrese letras en lugar de un número al registrar un precio). El código debe anticiparse a estas fallas mediante validaciones preventivas. Asimismo, las normas de seguridad en el laboratorio de cómputo exigen mantener el espacio libre de líquidos, evitar conexiones eléctricas sobrecargadas y respetar las políticas de respaldo de información.

A continuación, analiza los siguientes conceptos y complétalos en el cuaderno activo para consolidar tu comprensión teórica:

```
<!--workbook:table:cols=Elemento,Analogía,Concepto Técnico,Aplicación-->
```

```
| Frasco con etiqueta | Recipiente rotulado con nombre | Variable informática | Almacenar stock de un producto |
| Receta de cocina | Pasos secuenciales para preparar un platillo | Algoritmo / Código fuente | Automatizar el cálculo de cambio |
| Ayudante de cocina | Persona que ejecuta literal las órdenes | CPU / Procesador | Ejecutar operaciones matemáticas |
```

```
<!--workbook:lines:rows=6-->
```

Redacta con tus propias palabras por qué es peligroso para un negocio local en Puebla llevar el control de su inventario únicamente de memoria o en papel, y cómo la abstracción de variables resuelve este problema.

```
<!--workbook:drawing:height=180-->
```

Realiza un boceto esquemático de cómo imaginas que la memoria de una computadora almacena tres variables distintas (nombre del producto, precio y cantidad) utilizando la analogía de la cocina poblana.

3. Yo Hago: Demostración y Protocolo Guiado por el Docente

A continuación, desarrollaremos una demostración guiada completa, un ejemplo maestro diseñado bajo estándares industriales. Este script en Python simula el módulo inicial de registro y control de inventario para el negocio de Don Filemón en Francisco Z. Mena. Analizaremos cada línea de código minuciosamente y revisaremos su arquitectura mediante un diagrama textual.

Diagrama de Arquitectura del Sistema:

```
[Inicio del Programa] ---> [Definición de Variables Iniciales] ---> [Entrada Interactiva de Datos del Usuario] ---> [Procesamiento Lógico y Aritmético] --->
[Validación de Stock] ---> [Salida de Resultados en Pantalla] ---> [Fin del Programa]
```

A continuación se presenta el código fuente ejecutable completo y documentado:

```
<!--workbook:code:lines=20-->
```

```
python
# Programa: Modulo Base de Inventario Comunitario
# Autor: Estudiante de Educación Media Superior
# Contexto: Francisco Z. Mena, Puebla
# Descripción: Este programa registra un producto, calcula el valor total del inventario
# y valida si es necesario reabastecer stock.
```

```
print('=== SISTEMA DE GESTION DE INVENTARIO COMUNITARIO ===')
print('Ubicacion: Metlatoyuca, Francisco Z. Mena, Puebla')
print('-----')
```

1. Definición de variables iniciales con nomenclatura estandar (snake_case)

```
nombre_comercio = 'Abarrotes Don Filemon'
nombre_producto = 'Aceite Vegetal 1L'
precio_unitario = 35.50      # Tipo float (decimal)
cantidad_stock = 12         # Tipo int (entero)
limite_minimo_reabastecimiento = 5 # Tipo int (entero)
```

2. Mostrar estado actual del inventario

```
print('Comercio: ', nombre_comercio)
print('Producto registrado: ', nombre_producto)
print('Precio unitario: $', precio_unitario)
print('Unidades disponibles en bodega: ', cantidad_stock)
```

3. Simulación de una venta comunitaria de 4 unidades

```
print('\n--- Registrando venta de 4 unidades ---')
productos_vendidos = 4
cantidad_stock = cantidad_stock - productos_vendidos
```

4. Cálculo del valor total del inventario restante

```
valor_total_inventario = cantidad_stock * precio_unitario
```

5. Salida de resultados actualizados

```
print('Stock actualizado despues de la venta: ', cantidad_stock)
print('Valor monetario total en inventario: $', valor_total_inventario)
```

6. Estructura de decisión condicional para alerta de reabastecimiento

```
necesita_reabastecer = cantidad_stock <= limite_minimo_reabastecimiento
print('¿Se requiere solicitar nuevo pedido al proveedor?: ', necesita_reabastecer)
```

if necesita_reabastecer:

```
    print('ALERTA INDUSTRIAL: El stock esta por debajo del limite seguro. Realizar pedido.')
```

else:

```
    print('ESTADO OPTIMO: El inventario cuenta con suficientes unidades.')
```

```
print('-----')
```

```
print('Fin de la ejecucion del modulo.')
```

Explicación línea por línea del código maestro:

- Líneas 1-5: Comentarios de cabecera que documentan el propósito del script, siguiendo los estándares de la industria para facilitar la auditoría y mantenimiento del software.
- Líneas 7-9: Instrucciones `print()` que muestran mensajes estéticos en la terminal, situando geográficamente el software en Puebla y dando identidad al sistema.
- Líneas 12-16: Declaración de variables clave (`nombre\_comercio`, `nombre\_producto`, `precio\_unitario`, `cantidad\_stock`, `limite\_minimo\_reabastecimiento`). Cada una utiliza un tipo de dato adecuado (cadenas de texto, números decimales y enteros) y nombres descriptivos sin abreviaturas confusas.
- Líneas 19-22: Despliegue en pantalla de los valores iniciales almacenados en las variables para verificar el estado de partida del sistema.
- Líneas 25-27: Simulación de una transacción comercial. Se declara la variable `productos\_vendidos` y se actualiza el valor de `cantidad\_stock` mediante una operación aritmética de resta, sobrescribiendo el espacio de memoria con el nuevo valor.
- Línea 30: Cálculo del capital inmovilizado o disponible en bodega multiplicando las unidades restantes por el precio unitario.
- Líneas 33-34: Impresión de los resultados actualizados para que el comerciante visualice de forma inmediata el impacto de la venta.
- Líneas 37-43: Evaluación lógica mediante una variable booleana y una estructura condicional (`if-else`). Si las unidades en stock son menores o iguales al límite mínimo, el sistema emite una alerta industrial automatizada, previniendo el desabastecimiento en la comunidad.

4. Nosotros Hacemos: Práctica Guiada en Equipo

Ahora es momento de trabajar en equipo. Con la guía de tu profesor y colaborando con tus compañeros, realizarán una práctica colaborativa titulada 'Diseño del Módulo de Control de Registro de Herramientas para un Taller Mecánico en Puebla'.

Instrucciones para el equipo:

1. Reúnete en equipos de tres o cuatro integrantes.
2. Imaginen que un taller mecánico local en Francisco Z. Mena necesita controlar el préstamo y existencia de herramientas pesadas (llaves de cruz,

gatos hidráulicos, pinzas de presión).

3. Diseñen en su cuaderno activo una tabla donde definan al menos cinco variables que utilizaría un programa para controlar este taller, especificando su nombre industrial, su tipo de dato (entero, flotante, texto o booleano) y un ejemplo de valor inicial.
4. Escriban en equipo el pseudocódigo o los pasos lógicos que seguiría el programa si un mecánico toma prestada una herramienta y luego la devuelve.

```
<!--workbook:table:cols=Nombre de Variable,Tipo de Dato,Valor Inicial de Ejemplo,Propósito en el Taller-->
| nombre_herramienta | Cadena de texto (string) | 'Llave stilson 14 pulgadas' | Identificar el tipo de herramienta |
| total_unidades | Entero (int) | 8 | Conocer la cantidad física disponible |
| costo_reposicion | Flotante (float) | 450.00 | Calcular valor monetario de pérdida |
| herramienta_disponible | Booleano (bool) | True | Verificar si se puede prestar |
| codigo_taller | Cadena de texto (string) | 'TALLER-MEC-01' | Identificar la sucursal comunitaria |
```

```
<!--workbook:lines:rows=8-->
```

Redacten como equipo la justificación de por qué seleccionaron esos tipos de datos específicos y qué riesgos existirían en el taller mecánico si utilizaran un tipo de dato incorrecto (por ejemplo, guardar el costo de reposición como texto en lugar de número flotante).

```
<!--workbook:drawing:height=160-->
```

Dibujen un diagrama de flujo colaborativo que represente el proceso de préstamo de una herramienta en el taller mecánico.

5. Tú Haces: Reto Autónomo de Aplicación

Ha llegado el momento de demostrar tu dominio autónomo absoluto mediante el reto individual 'Mi Primer Script de Inventario para Frutería Comunitaria'.

Contexto del Reto:

Imagina que una frutería comunitaria en Metlatoyuca, Puebla, te ha contratado para crear un mini programa en Python que calcule el inventario diario de limones. Debes escribir un script completo que cumpla con los siguientes requisitos industriales obligatorios:

1. Declarar al menos cuatro variables con nombres descriptivos en snake_case y tipos de datos correctos (`nombre\_fruta`, `precio\_kilo`, `kilos\_disponibles`, `fruta\_fresca`).
2. Simular la venta de una cantidad específica de kilos de limones restándola del inventario total.
3. Calcular el ingreso total obtenido por esa venta multiplicando los kilos vendidos por el precio por kilo.
4. Utilizar una estructura condicional (`if-else`) para verificar si los kilos disponibles después de la venta son menores a 10 kilos; si es así, mostrar un mensaje de alerta indicando que es necesario surtir más fruta.
5. Incluir comentarios detallados al inicio de cada bloque explicando tu código.

Escribe tu código autónomo en el siguiente espacio:

```
<!--workbook:code:lines=18-->
```

```
python
# Reto Autonomo: Control de Inventario de Fruteria en Puebla
# Alumno: [Tu Nombre]
# Contexto: Francisco Z. Mena, Puebla
```

```
print('=== CONTROL DE INVENTARIO: FRUTERIA COMUNITARIA ===')
```

```
# 1. Declaracion de variables con tipos de datos correctos
```

```
nombre_fruta = 'Limon Persa'
```

```
precio_kilo = 22.50      # Tipo float
```

```
kilos_disponibles = 45.0    # Tipo float
fruta_fresca = True        # Tipo booleano
```

```
# 2. Despliegue de estado inicial
print('Producto: ', nombre_fruta)
print('Precio por kilo: $', precio_kilo)
print('Kilos disponibles en bodega: ', kilos_disponibles)
```

```
# 3. Simulacion de venta de 15.5 kilos
kilos_vendidos = 15.5
kilos_disponibles = kilos_disponibles - kilos_vendidos
```

```
ingreso_venta = kilos_vendidos * precio_kilo
```

```
# 4. Resultados de la transaccion
print('\nKilos restantes despues de la venta: ', kilos_disponibles)
print('Ingreso total generado por la venta: $', ingreso_venta)
```

```
# 5. Estructura condicional de alerta de stock
alerta_surtir = kilos_disponibles < 10.0
```

```
if alerta_surtir:
    print('ALERTA: Quedan pocos kilos. Urge surtir nueva fruta.')
else:
    print('ESTABILIDAD: Inventario suficiente en la fruteria.')
```

```
print('=== FIN DEL PROCESO DE INVENTARIO ===')
```

<!--workbook:lines:rows=6-->

Escribe una breve conclusión personal (máximo 5 renglones) sobre cómo este pequeño programa tecnológico podría ayudar a un comerciante local en Puebla a evitar pérdidas económicas.

7. Punto de Control Formativo (Metacognición)

Pregunta formativa: ¿De qué manera la correcta selección de los tipos de datos y el uso de variables con nombres descriptivos (estándar industrial) garantizan que un software desarrollado para comercios en Puebla sea confiable y fácil de mantener?

- ¿Cómo se relaciona la analogía de la cocina poblana con la forma en que la memoria de una computadora almacena y manipula datos?
- ¿Qué importancia tienen las normas de seguridad y buenas prácticas al programar soluciones tecnológicas para nuestra comunidad?

Criterios de Verificación:

- [] Comprende y explica con claridad la analogía física de variables y algoritmos.
- [] Identifica y utiliza correctamente los tipos de datos primitivos en código ejecutable.
- [] Aplica normas de nomenclatura industrial (snake_case) y documentación en sus scripts.
- [] Diseña soluciones lógicas orientadas a resolver problemáticas reales de comercio en Puebla.

MISIÓN 2: LABORATORIO Y TALLER EXPERIMENTAL — ANÁLISIS DE ESPECIFICACIONES TÉCNICAS Y

Sesiones asignadas: 4, 5, 6, 7, 8, 9 | Enfoque: Análisis de especificaciones técnicas y procedimientos (Etapa 1) · Práctica de taller: Ejecución y destreza

1. Enganche y Desafío Situado en la Comunidad

Durante esta práctica en el aula o taller, abordaremos directamente cómo manipular variables operativas de Programación para resolver retos vinculados a Desarrollo de software y automatización digital comunitaria para comercios y talleres locales de Francisco Z. Mena, mitigando la brecha digital y optimizando el inventario comunitario..

Pregunta Detonadora: ¿De qué forma la precisión en la recolección de datos y la ejecución determina la confiabilidad del producto final?

2. Concepto Cero: Analogía Intuitiva y Fundamento

Analogía Física Cotidiana: Cada medición experimental es como un mapa de navegación: un error de un milímetro en el punto de partida te lleva a kilómetros del destino.

Analizar rigurosamente las especificaciones técnicas, requerimientos funcionales y no funcionales, así como el diseño algorítmico modular para el desarrollo de un sistema de control de inventarios automatizado enfocado en mitigar la brecha digital y optimizar la gestión de comercios y talleres locales en Francisco Z. Mena, Puebla, aplicando buenas prácticas de programación estructurada, validación estricta de datos y control de excepciones en Python.

3. Yo Hago: Demostración y Protocolo Guiado por el Docente

Protocolo Técnico:

1. Preparación, configuración y verificación del entorno de desarrollo de software.

El primer paso crítico en cualquier ciclo de desarrollo de software profesional bajo el marco de la Nueva Escuela Mexicana (NEM) y el MCCEMS es asegurar la estandarización y correcta configuración del entorno de trabajo. Esto evita discrepancias entre equipos de cómputo y garantiza que la ejecución de los algoritmos sea idéntica en cualquier plataforma. Para iniciar, encienda su estación de trabajo, verifique que los periféricos funcionen adecuadamente y abra una terminal de comandos (PowerShell en Windows o Bash en entornos Unix). Verifique la versión exacta del intérprete de Python ejecutando el comando 'python --version' o 'python3 --version'. El sistema debe retornar una versión igual o superior a 3.10.0. En caso de discrepancias, notifique al instructor técnico de inmediato. A continuación, cree un directorio de trabajo dedicado para esta práctica utilizando el comando 'mkdir mision_2_inventarios' y acceda a él mediante 'cd mision_2_inventarios'. Abra Visual Studio Code desde esta terminal utilizando el comando 'code .' . Una vez abierto el editor, cree un archivo virtual de entorno con el comando 'python -m venv venv' y actívelo mediante 'source venv/bin/activate' en Linux/macOS o '.\venv\Scripts\activate' en Windows. Esta práctica aísla las dependencias del proyecto y garantiza la reproducibilidad técnica exigida en los protocolos industriales y académicos. Registre sus observaciones iniciales en el espacio asignado en su cuaderno activo y verifique que la extensión de Pylint esté activa y configurada para detectar errores de sintaxis y estilo PEP 8 en tiempo real.

2. Análisis e interpretación de requerimientos funcionales y no funcionales del FRS (Functional Requirements Specification).

En este paso nos adentraremos en la lectura crítica y descomposición técnica del documento de Especificación de Requerimientos de Software (FRS) diseñado para los pequeños comercios y talleres locales de Francisco Z. Mena (abarrotes, talleres mecánicos, papelerías y ferreterías comunitarias). La problemática PAEC exige desarrollar un sistema digital que resuelva el descontrol de mercancías, el registro manual propenso a errores humanos y la falta de alertas de stock mínimo. Lea detenidamente el FRS proporcionado en el anexo técnico. Identifique y desglose en su cuaderno activo al menos tres requerimientos funcionales críticos (por ejemplo: el sistema debe permitir el registro de altas, bajas y modificaciones de productos; el sistema debe calcular automáticamente el valor total del inventario en almacén; el sistema debe generar reportes de productos con stock crítico menor a cinco unidades) y dos requerimientos no funcionales (por ejemplo: el tiempo de respuesta en consultas de inventario no debe superar los 500 milisegundos; la interfaz de consola debe ser intuitiva y presentar un diseño estructurado mediante tablas de texto formateado). Comprender estos requerimientos antes de escribir una sola línea de código es la diferencia entre un software robusto y un desarrollo efímero. Documente la matriz de trazabilidad preliminar en la siguiente tabla de análisis.

<!--workbook:lines:rows=5-->

3. Diseño del diagrama de flujo lógico y arquitectura modular de funciones.

Antes de programar, es una norma de ingeniería de software estructurar el flujo de control mediante pseudocódigo o diagramas de flujo. En este paso, el estudiante dibujará en su cuaderno de laboratorio el diagrama de flujo correspondiente al módulo central de control de inventario del comercio local. El sistema debe operar bajo un menú interactivo en bucle infinito controlado por una bandera booleana o una opción de salida explícita. El menú principal ofrecerá cuatro opciones obligatorias: 1) Registrar nuevo producto, 2) Consultar inventario actual, 3) Actualizar stock de producto existente, y 4) Salir del sistema. Cada opción debe estar representada por una función modular independiente en Python para cumplir con el principio de responsabilidad única (Single Responsibility Principle). Por ejemplo, la función 'registrar_producto(inventario)' recibirá como parámetro la estructura de datos principal (un diccionario o lista de diccionarios) y solicitará al usuario mediante entradas de consola ('input') el código del producto, la

descripción, el precio unitario y la cantidad inicial en stock. Analice cómo se comportarán las estructuras de datos ante entradas vacías o caracteres alfabéticos cuando se espera un valor numérico. Trace el diagrama de flujo completo contemplando las rutas verdaderas y falsas para las validaciones de entrada.

```
<!--workbook:lines:rows=5-->
```

4. Implementación de la estructura de datos base y carga inicial de simulación comunitaria.

Proceda a crear un nuevo archivo de código fuente en su entorno de trabajo con el nombre 'sistema_inventario_paec.py'. En este paso inicializaremos la estructura de datos que simulará la base de datos local del comercio comunitario. Para optimizar el rendimiento y la legibilidad en Python, utilizaremos una lista de diccionarios, donde cada diccionario representa un artículo con las siguientes llaves estandarizadas: 'codigo' (cadena de texto alfanumérica única), 'nombre' (cadena de texto), 'precio' (número flotante positivo) y 'stock' (número entero no negativo). Escriba el bloque de código inicial que declare una lista llamada 'inventario_comunitario' precargada con al menos tres productos típicos de la región de Francisco Z. Mena (por ejemplo: 'Hules para empaque agrícola', 'Clavos de acero 2 pulgadas', 'Aceite lubricante multigrado'). Comente cada línea explicando la razón técnica de la estructura de datos elegida en comparación con listas paralelas o matrices bidimensionales. Verifique que la ejecución inicial del script no arroje errores de sintaxis al declarar los tipos de datos.

```
<!--workbook:lines:rows=4-->
```

5. Desarrollo y codificación de la función de validación robusta de entradas numéricas y de texto.

Uno de los errores más comunes en los sistemas desarrollados para la gestión comunitaria es la caída abrupta del software ('crashes') debido a que el usuario ingresa letras cuando el sistema espera un número flotante o un entero. En este paso, programaremos una función robusta de entrada de datos llamada 'obtener_flotante_seguro(mensaje)' y 'obtener_entero_seguro(mensaje)'. Estas funciones utilizarán la estructura de control de excepciones 'try-except' de Python. Si el usuario introduce un valor no válido (por ejemplo, introducir la palabra 'cinco' en lugar del número '5'), el bloque 'except ValueError' interceptará el error, mostrará una advertencia clara en pantalla indicando el formato correcto y solicitará nuevamente el dato mediante un bucle 'while True' hasta que se reciba un valor válido. Esta técnica garantiza la robustez y resiliencia del software ante operadores con poca experiencia digital. Escriba y pruebe la función en su archivo de código, documentando el comportamiento del bloque de excepciones.

```
<!--workbook:lines:rows=4-->
```

6. Codificación de la función modular para el registro de altas de nuevos productos.

Implemente la función 'registrar_producto(inventario)' en su script. Esta función debe solicitar al usuario el código único del producto. Antes de agregarlo a la lista, el algoritmo debe recorrer el inventario existente mediante un bucle de búsqueda para verificar que el código no esté duplicado; si ya existe, debe rechazar el registro y notificar al usuario. Si el código es único, procederá a solicitar el nombre, precio unitario y stock inicial utilizando las funciones de validación robustas desarrolladas en el paso anterior. Una vez recolectados y validados todos los campos, se empaquetará la información en un nuevo diccionario y se añadirá a la lista principal utilizando el método '.append()'. Imprima un mensaje de confirmación formateado que muestre los datos del producto recién registrado. Registre en su cuaderno las pruebas realizadas con códigos duplicados y valores erróneos.

```
<!--workbook:lines:rows=4-->
```

7. Codificación de la función de consulta y despliegue tabular de inventario.

Desarrolle la función 'consultar_inventario(inventario)'. Esta función tiene la responsabilidad de recorrer la lista de diccionarios y mostrar todos los productos registrados de manera ordenada, limpia y estructurada, imitando una tabla profesional de base de datos en la consola de comandos. Utilice operadores de formato de cadenas en Python (f-strings con alineación y especificadores de ancho, por ejemplo: '{codigo:<10} | {nombre:<25} | \${precio:>8.2f} | {stock:>6} unidades'). Si el inventario está vacío, la función debe interceptar esta condición mediante una sentencia condicional 'if len(inventario) == 0:' y mostrar un aviso amigable informando que no existen registros actuales y sugiriendo dar de alta el primer producto. Calcule además el valor total monetario sumando el producto de 'precio * stock' de cada artículo e imprima el gran total del capital invertido en mercancía al pie de la tabla.

```
<!--workbook:lines:rows=4-->
```

8. Codificación de la función de actualización de stock y control de mermas o reabastecimiento.

Implemente la función 'actualizar_stock(inventario)'. En un entorno comercial real, el inventario cambia constantemente debido a ventas, mermas o llegada de nueva mercancía desde proveedores regionales. Esta función solicitará al usuario el código del producto a modificar. Mediante una búsqueda secuencial, localizará el diccionario correspondiente. Una vez encontrado, mostrará el nombre actual y el stock disponible, preguntando al usuario si desea sumar stock (reabastecimiento) o restar stock (venta o merma), o bien reemplazar el valor absoluto. Utilice validaciones estrictas para asegurar que el stock resultante nunca sea menor a cero. Si el stock resultante cae por debajo de un umbral crítico de alerta (por ejemplo, menor o igual a 3 unidades), el sistema debe emitir un mensaje destacado de advertencia en color o con texto resaltado indicando '¡ALERTA: STOCK CRÍTICO!'. Documente en su cuaderno los resultados de las pruebas de actualización.

```
<!--workbook:lines:rows=4-->
```

9. Integración del ciclo principal de control (Main Loop) y manejo de salidas del sistema.

En este paso uniremos todos los módulos desarrollados en una estructura de control principal utilizando un bloque 'if __name__ == \'__main__\':'. Cree una función 'menu_principal()' que gestione el flujo general mediante un bucle 'while True'. El menú debe mostrar claramente las opciones numeradas. Utilice una estructura condicional 'if-elif-else' para direccionar la ejecución hacia la función correspondiente según la opción seleccionada por el usuario. Agregue la opción de salida (por ejemplo, opción 4 o 5) que ejecute un comando 'break' para romper el ciclo y despedirse amigablemente del usuario, asegurando que antes de cerrar se imprima un resumen de la sesión de trabajo. Pruebe la integración completa de todo el sistema

navegando por todas las opciones del menú de forma consecutiva y sin interrupciones.

```
<!--workbook:lines:rows=4-->
```

10. Pruebas de caja blanca, depuración de errores sintácticos y validación lógica.

Una vez integrado el código completo, es momento de realizar una auditoría de caja blanca. Revise línea por línea la indentación, el nombramiento de variables (asegurando el cumplimiento de la norma PEP 8 en español o inglés técnico estandarizado), la correcta apertura y cierre de paréntesis, y la ausencia de variables huérfanas que no se utilicen en el programa. Utilice el depurador integrado de Visual Studio Code (Debug mode) colocando puntos de interrupción ('breakpoints') en las líneas donde se procesan los cálculos de inventario y actualización de stock. Ejecute el programa paso a paso ('Step Over', 'Step Into') y observe en el panel lateral de variables cómo cambia el estado de la lista de diccionarios en tiempo real. Registre cualquier anomalía o desfase lógico detectado y corrija el código inmediatamente.

```
<!--workbook:lines:rows=4-->
```

11. Pruebas de caja negra y simulación de escenarios de uso comunitario real.

Simule el rol de un comerciante de Francisco Z. Mena utilizando el sistema operativo de consola recién programado. Ejecute pruebas de caja negra ingresando casos extremos: intente registrar un producto con precio negativo, ingrese un código con espacios en blanco y caracteres especiales, intente actualizar el stock de un producto que no existe en la base de datos, y vacíe el inventario por completo para verificar la respuesta ante listas vacías. Cada uno de estos escenarios debe ser manejado limpiamente por el sistema mediante las excepciones programadas, sin mostrar errores largos en lenguaje de máquina ni cerrar abruptamente la aplicación. Anote los resultados de cada prueba de caja negra en la tabla de datos experimentales de su cuaderno activo.

```
<!--workbook:lines:rows=4-->
```

12. Documentación técnica del código, elaboración de manual de usuario y cierre de la práctica.

Como paso final de esta primera etapa de laboratorio, redacte una documentación técnica concisa dentro del mismo archivo de código mediante comentarios de bloque ("""...""") explicando la arquitectura del sistema, los parámetros de entrada y salida de cada función, y las restricciones operativas. Asimismo, redacte un mini manual de usuario de tres párrafos dirigido al tendero o artesano local, explicando de forma sencilla cómo encender el sistema, cómo registrar su mercancía y qué hacer en caso de que aparezca una alerta de stock crítico. Guarde todos los archivos en su repositorio local, realice una última revisión contra la rúbrica de evaluación y entregue su cuaderno de notas y el archivo de código fuente al instructor técnico para su validación oficial.

4. Nosotros Hacemos: Práctica Guiada en Equipo

Desarrollo guiado en equipo:

```
"""
```

UAC: Programación (BT - DBEPA / MCCEMS)

Misión 2: Laboratorio y Taller Experimental - Análisis de especificaciones técnicas y procedimientos (Etapa 1)

Descripción: Sistema automatizado de control de inventarios para comercios y talleres comunitarios en Francisco Z. Mena.

Autor: Estudiante de Bachillerato Técnico

Fecha: 2024

```
"""
```

```
def obtener_flotante_seguro(mensaje):
```

```
    """
```

Función para validar la entrada de datos numéricos flotantes (precios, costos).

Previene caídas del sistema por ingresos erróneos de texto.

```
    """
```

```
    while True:
```

```
        try:
```

```
            valor = float(input(mensaje))
```

```
            if valor < 0:
```

```
                print('Error: El valor no puede ser negativo. Intente nuevamente.')
```

```
                continue
```

```
            return valor
```

```
        except ValueError:
```

```
            print('Error de formato: Ingrese un número válido (ejemplo: 150.50).')
```

```
def obtener_entero_seguro(mensaje):
```

```
    """
```

Función para validar la entrada de datos numéricos enteros (stock, cantidades).

Previene caídas del sistema por ingresos de decimales o letras.

```
    """
```

```
    while True:
```

```
        try:
```

```

    valor = int(input(mensaje))
    if valor < 0:
        print('Error: La cantidad no puede ser negativa. Intente nuevamente.')
        continue
    return valor
except ValueError:
    print('Error de formato: Ingrese un número entero válido (ejemplo: 12).')

def registrar_producto(inventario):
    """
    Módulo de alta de nuevos productos en el inventario comunitario.
    Verifica unicidad de código y solicita datos validados.
    """
    print("\n--- REGISTRO DE NUEVO PRODUCTO ---")
    codigo = input('Ingrese el código alfanumérico único del producto: ').strip()

    # Validación de código duplicado
    for producto in inventario:
        if producto['codigo'] == codigo:
            print(f'Error crítico: El código {codigo} ya está registrado en el sistema.')
            return

    nombre = input('Ingrese el nombre descriptivo del producto: ').strip()
    if not nombre:
        print('Error: El nombre del producto no puede estar vacío.')
        return

    precio = obtener_flotante_seguro('Ingrese el precio unitario ($): ')
    stock = obtener_entero_seguro('Ingrese la cantidad inicial en stock: ')

    nuevo_producto = {
        'codigo': codigo,
        'nombre': nombre,
        'precio': precio,
        'stock': stock
    }

    inventario.append(nuevo_producto)
    print(f'Éxito! Producto \"{nombre}\" registrado correctamente en el inventario.')

def consultar_inventario(inventario):
    """
    Módulo de consulta y despliegue tabular del inventario actual.
    Calcula el valor total monetario almacenado.
    """
    print("\n--- INVENTARIO GENERAL DE COMERCIO LOCAL ---")
    if len(inventario) == 0:
        print('El inventario se encuentra actualmente vacío. Registre productos para comenzar.')
        return

    print(f'{"\tCODIGO\t":<10} | {"\tDESCRIPCION PRODUCTO\t":<25} | {"\tPRECIO\t":>10} | {"\tSTOCK\t":>8} | {"\tSUBTOTAL\t":>12}')
    print('-' * 75)

    valor_total_inventario = 0.0
    for p in inventario:
        subtotal = p['precio'] * p['stock']
        valor_total_inventario += subtotal
        print(f'{"\tp[codigo]:<10} | "\tp[nombre]:<25} | ${p[precio]:>9.2f} | {p[stock]:>8} | ${subtotal:>11.2f}"')

    print('-' * 75)
    print(f'VALOR TOTAL DEL INVENTARIO EN ALMACÉN: ${valor_total_inventario:.2f}\n')
```

```

def actualizar_stock(inventario):
    """
    Módulo de actualización de stock (reabastecimiento o venta).
    Incluye alertas automáticas de stock crítico.
    """
    print("\n--- ACTUALIZACIÓN DE STOCK Y EXISTENCIAS ---")
    if len(inventario) == 0:
        print("No hay productos registrados para actualizar.")
        return

    codigo = input("Ingrese el código del producto a actualizar: ").strip()
    producto_encontrado = None

    for p in inventario:
        if p['codigo'] == codigo:
            producto_encontrado = p
            break

    if producto_encontrado is None:
        print(f"Error: No se encontró ningún producto con el código {codigo}.")
        return

    print(f"Producto seleccionado: {producto_encontrado['nombre']} | Stock actual: {producto_encontrado['stock']}")
    print("Seleccione la operación a realizar:")
    print("1. Sumar stock (Reabastecimiento)")
    print("2. Restar stock (Venta o Merma)")
    print("3. Reemplazar stock por valor absoluto")

    opcion = obtener_entero_seguro("Ingrese opción de actualización (1-3): ")

    if opcion == 1:
        cantidad = obtener_entero_seguro("Ingrese la cantidad a agregar: ")
        producto_encontrado['stock'] += cantidad
        print(f"Stock actualizado con éxito. Nuevo stock: {producto_encontrado['stock']}")
    elif opcion == 2:
        cantidad = obtener_entero_seguro("Ingrese la cantidad a descontar: ")
        if cantidad > producto_encontrado['stock']:
            print("Error: No se puede descontar una cantidad mayor al stock disponible.")
        else:
            producto_encontrado['stock'] -= cantidad
            print(f"Stock actualizado con éxito. Nuevo stock: {producto_encontrado['stock']}")
    elif opcion == 3:
        cantidad = obtener_entero_seguro("Ingrese el nuevo stock absoluto: ")
        producto_encontrado['stock'] = cantidad
        print(f"Stock actualizado con éxito. Nuevo stock: {producto_encontrado['stock']}")
    else:
        print("Opción no válida. Operación cancelada.")
        return

    # Verificación de alerta de stock crítico (umbral menor o igual a 3 unidades)
    if producto_encontrado['stock'] <= 3:
        print(f"*** ¡ALERTA COMUNITARIA: STOCK CRÍTICO! El producto '{producto_encontrado['nombre']}' tiene {producto_encontrado['stock']} unidades restantes. Reabastézca pronto. ***")

def menu_principal():
    """
    Función principal que controla el flujo de ejecución del software comunitario.
    """
    # Carga inicial de simulación comunitaria (Francisco Z. Mena)
    inventario_comunitario = [

```

```
{'codigo': 'AGR01', 'nombre': 'Hules Empaque Agrícola', 'precio': 45.50, 'stock': 12},  
{'codigo': 'HER02', 'nombre': 'Clavos Acero 2 Pulgadas', 'precio': 38.00, 'stock': 2},  
{'codigo': 'LUB03', 'nombre': 'Aceite Multigrado 4T', 'precio': 125.00, 'stock': 5}  
]
```

while True:

```
print("\n=====")  
print(' SISTEMA DE GESTIÓN DE INVENTARIOS - PAEC Z. MENA ')  
print("=====")  
print('1. Registrar nuevo producto')  
print('2. Consultar inventario y valor total')  
print('3. Actualizar stock (Entrada / Salida)')  
print('4. Salir del sistema')
```

```
opcion = obtener_entero_seguro('Seleccione una opción del menú (1-4): ')
```

```
if opcion == 1:  
    registrar_producto(inventario_comunitario)  
elif opcion == 2:  
    consultar_inventario(inventario_comunitario)  
elif opcion == 3:  
    actualizar_stock(inventario_comunitario)  
elif opcion == 4:  
    print("\nGracias por utilizar el sistema de automatización comunitaria.")  
    print('Cerrando sesión de trabajo de forma segura. ¡Hasta pronto!')  
    break  
else:  
    print("Error: Opción fuera de rango. Por favor seleccione un número entre 1 y 4.")
```

```
if __name__ == '__main__':  
    menu_principal()
```

5. Tú Haces: Reto Autónomo de Aplicación

Modifique el script en Python desarrollado en esta práctica para incorporar un nuevo submódulo modular llamado 'eliminar_producto(inventario)'. Este submódulo debe solicitar al usuario el código alfanumérico del artículo que desea dar de baja definitiva del inventario comunitario de Francisco Z. Mena. El algoritmo debe realizar una búsqueda exhaustiva del elemento en la lista de diccionarios. Antes de ejecutar la eliminación física mediante el método '.pop()' o la instrucción 'del', el programa debe mostrar por consola todos los detalles del producto encontrado (código, nombre, precio y stock actual) y exigir una confirmación explícita de seguridad preguntando: '¿Está seguro de eliminar este producto del sistema comunitario? (s/n): '. Si el usuario ingresa 's' o 'S', se procederá a eliminar el registro y se imprimirá un mensaje de confirmación de baja exitosa; si el usuario ingresa 'n' o cualquier otro carácter, la operación se cancelará de inmediato. Asimismo, asegúrese de que si el código ingresado no existe en la base de datos, el sistema maneje el caso elegantemente informando al operador sin generar excepciones no controladas. Integre esta nueva opción en el menú principal reenumerando las opciones correspondientes de manera coherente y documente sus pruebas en el cuaderno activo.

6. Matriz de Resiliencia: "¿Qué hacer si falla?"

Síntoma / Falla	Causa Raíz	Solución Metódica	Prevención
La terminal arroja el error 'SyntaxError: invalid syntax' al ejecutar el script de Python en el laboratorio.	Falta de un carácter de cierre (como paréntesis, comillas o dos puntos) en la línea de código señalada por el intérprete.	Revisa el número de línea indicado en el error. • Verifica que todas las estructuras de control (if, for, def) terminen con dos puntos (:). • Comprueba que cada paréntesis o comilla abierta tenga su respectivo cierre.	Utiliza un editor de código con resaltado de sintaxis y autocompletado habilitado antes de ejecutar las pruebas.
Al conectar el circuito con el microcontrolador y el sensor al protoboard, el LED indicador no enciende y se percibe olor a sobrecalentamiento.	Inversión en la polaridad de la fuente de alimentación (cables de VCC y GND invertidos) o cortocircuito directo en los Rieles del protoboard.	Desconecta inmediatamente la fuente de alimentación USB o externa. • Revisa el código de colores: cable rojo a positivo y negro o azul a negativo. • Verifica con el multímetro la continuidad antes de energizar de nuevo.	Cumple estrictamente con la NOM-001-SEDE sobre instalaciones eléctricas y revisa el diagrama de conexiones con el docente antes de energizar.
El programa compila, pero al introducir un número decimal el sistema se detiene abruptamente con 'ValueError: invalid literal for int()'.	Se está intentando convertir una entrada de texto flotante directamente a un entero utilizando la función int() en lugar de float().	Localiza la línea donde se captura la entrada del usuario. • Modifica la conversión de int(input()) a float(input()). • Viste la variable con el tipo de dato correcto para que acepte decimales.	Analiza las especificaciones técnicas y el tipo de datos requeridos en el análisis previo antes de codificar la lógica de entrada.
El multímetro digital marca cero o muestra 'OL' (Over Limit) al intentar medir el voltaje en los pines de salida del circuito lógico.	Las puntas del multímetro están conectadas en los bornes incorrectos (por ejemplo, midiendo voltaje con las puntas en la terminal de corriente) o la escala seleccionada es errónea.	Asegúrate de que la punta roja esté en el borne de voltaje/resistencia (VⓈmA) y la negra en el de tierra (COM). • Gira la perilla selectora al rango adecuado de voltaje directo (V DC). • Vuelve a realizar la medición en paralelo.	Realiza una inspección visual del instrumental y calibra los instrumentos de medición siguiendo el manual de procedimientos antes de iniciar la práctica.
El control de calidad automatizado arroja resultados inconsistentes y las pruebas de funcionamiento fallan de forma aleatoria.	Falsos contactos en el cableado del protoboard debido a cables con terminales dañadas o mal insertadas en los n	Apaga el sistema. • Revisa uno a uno los puentes y conexiones en el protoboard asegurando que hagan contacto firme. • Sustituye los cables que presenten dobleces excesivos o terminales flojas.	Implementa buenas prácticas de armado manteniendo orden y utilizando componentes en óptimas condiciones durante la Etapa 1 y 7.
El entorno de desarrollo integrado (IDE) marca 'Error: Port busy' o no detecta la placa de desarrollo conectada por puerto USB.	El puerto COM asignado está siendo utilizado por otra aplicación de fondo o el controlador (driver) del dispositivo no está instalado correctamente.	Cierra otras instancias de software de programación o terminales abiertas. • Desconecta y vuelve a conectar el cable USB. • Verifica en el Administrador de Dispositivos el puerto COM correcto y selecciónalo en el IDE.	Verifica la compatibilidad de drivers y el estado de los puertos de comunicación antes de iniciar la sesión de laboratorio.

7. Punto de Control Formativo (Metacognición)

Pregunta formativa: ¿Los valores experimentales coincidieron con los calculados teóricamente? Justifica tu respuesta.

- ¿Qué relación directa observaste entre el diseño modular de funciones en Python (separar el registro, la consulta y la actualización) y la mantenibilidad del código ante futuros cambios requeridos por los comerciantes de Francisco Z. Mena?
- ¿Cómo influyó el uso de la estructura de control de excepciones (try-except) en la prevención de fallos críticos del software cuando un usuario ingresaba caracteres alfabéticos en lugar de valores numéricos de stock o precio?
- ¿De qué manera la implementación de un algoritmo de búsqueda por código único previene la duplicidad de registros y garantiza la integridad de los datos en el inventario comunitario del proyecto PAEC?
- ¿Qué implicaciones éticas y sociales tiene el desarrollo de software local adaptado a las necesidades específicas de los pequeños talleres y comercios rurales frente al uso de software comercial genérico o privativo?
- ¿Cómo aplicarías este mismo protocolo de análisis de especificaciones técnicas y depuración estructurada para resolver una problemática de automatización en otra área de tu comunidad o plantel educativo?

Criterios de Verificación:

- [] Registré todas las lecturas con sus unidades de medida correspondientes.

MISIÓN 3: CONSTRUCCIÓN DEL ARTEFACTO REAL — ANÁLISIS DE ESPECIFICACIONES TÉCNICAS Y P

Sesiones asignadas: 10, 11, 12, 13, 14, 15 | Enfoque: Construcción del artefacto e integración socioproductiva

1. Enganche y Desafío Situado en la Comunidad

Llegamos a la fase de transferencia y aplicación: aquí transformamos el conocimiento de Programación en un artefacto real que atiende la problemática de Desarrollo de software y automatización digital comunitaria para comercios y talleres locales de Francisco Z. Mena, mitigando la brecha digital y optimizando el inventario comunitario..

Pregunta Detonadora: ¿De qué manera una solución técnica bien diseñada puede mejorar la calidad de vida en nuestro entorno escolar y comunitario?

2. Concepto Cero: Analogía Intuitiva y Fundamento

Analogía Física Cotidiana: Un proyecto es como construir un puente: no basta con que los cálculos sean elegantes en papel, debe soportar el peso de las personas que lo cruzan.

El desarrollo del artefacto tecnológico denominado 'Sistema Integral de Control de Inventario y Ventas para Talleres y Comercios Locales (SICIV-FZM)' se enmarca de manera rigurosa dentro de la estrategia de la Nueva Escuela Mexicana (NEM) y del Plan de Acción Educativo Comunitario (PAEC) para el municipio de Francisco Z. Mena, Puebla. Este municipio, caracterizado por una economía local basada en pequeños comercios, talleres mecánicos, carpinterías, misceláneas y microempresas familiares, enfrenta una marcada brecha digital y tecnológica que limita su crecimiento económico, su capacidad de organización administrativa y su inserción en mercados competitivos. La gran mayoría de estos establecimientos tradicionales operan mediante métodos rudimentarios de registro manual, utilizando libretas de papel, notas de remisión físicas y cálculos aritméticos manuales o memorísticos. Esta situación genera una serie de problemáticas sistémicas graves: pérdidas financieras por descontrol en las existencias de mercancía, merma de productos perecederos no detectada a tiempo, desconocimiento de los artículos con mayor o menor rotación (productos 'hueso'), errores frecuentes en el cálculo de precios y utilidades, falta de visibilidad sobre los flujos de efectivo diarios, y una enorme vulnerabilidad ante robos, extravíos o daños físicos de los registros contables. Ante este escenario socioeconómico, la Unidad de Aprendizaje Curricular (UAC) de Programación, correspondiente al subsistema de Bachillerato Tecnológico (BT), asume un compromiso ético, social y técnico ineludible. No se trata de un ejercicio académico aislado dentro de un laboratorio de cómputo, sino de una intervención tecnológica real y transformadora. Los estudiantes de educación media superior se convierten en ingenieros de software comunitarios, diseñadores de soluciones digitales y agentes de cambio social que responden directamente a las necesidades de su entorno inmediato. El artefacto que se construye consiste en una aplicación de software ejecutable de escritorio y/o web ligera, robusta y accesible, desarrollada con lenguajes de programación estructurados y orientados a objetos (como Python, JavaScript o Java, respaldados por bases de datos relacionales como SQLite o MySQL), diseñada específicamente para adaptarse a las capacidades de hardware existentes en los pequeños comercios de Francisco Z. Mena (computadoras de escritorio de generaciones anteriores, laptops recicladas o terminales de bajo costo). La pertinencia técnica del SICIV-FZM radica en su arquitectura modular y su interfaz de usuario intuitiva, diseñada bajo principios de usabilidad universal para personas adultas con poca o nula familiaridad previa con herramientas informáticas. El sistema resuelve de forma integral tres ejes operativos críticos para cualquier comercio local: 1) Control automatizado de inventarios en tiempo real, con alertas visuales de stock mínimo para evitar el desabastecimiento y la pérdida de ventas; 2) Módulo de punto de venta (POS) ágil, que permite registrar transacciones de manera rápida mediante códigos de barras generados localmente o códigos numéricos estandarizados, emitiendo tickets digitales o listados imprimibles; y 3) Generación de reportes gerenciales automatizados, que muestran al locatario de manera gráfica y sencilla cuáles son los productos más vendidos, las ganancias netas por periodo de tiempo, y el valor total del inventario acumulado. Los beneficiarios directos de este proyecto son los propietarios y trabajadores de los talleres y comercios locales de Francisco Z. Mena, quienes verán optimizado su tiempo administrativo en hasta un 70%, reduciendo drásticamente los errores humanos en el manejo de efectivo y mercancías. Los beneficiarios indirectos incluyen a las familias de los comerciantes, cuya estabilidad económica depende del éxito del negocio, y a la comunidad en general, que se beneficia de un comercio local más formal, organizado, competitivo y capaz de ofrecer un servicio más eficiente. Desde la perspectiva pedagógica del Marco Curricular Común de la Educación Media Superior (MCCEMS), este artefacto propicia el desarrollo de competencias disciplinares extendidas en el campo de las ciencias exactas y la tecnología, así como competencias profesionales de la carrera técnica en programación. El estudiante moviliza saberes complejos relacionados con el análisis de requerimientos, el diseño de bases de datos normalizadas, la programación de algoritmos de búsqueda y ordenamiento, la validación de entradas de usuario, la gestión de excepciones y la depuración de código. Asimismo, fomenta habilidades socioemocionales fundamentales como la empatía comunitaria, la escucha activa para comprender las necesidades reales del usuario final, la responsabilidad profesional, el trabajo en equipo colaborativo y la resiliencia ante la resolución de bugs y fallas técnicas durante el despliegue del software. En conclusión, el SICIV-FZM no es meramente un programa informático que cumple con una rúbrica escolar; es una herramienta de emancipación tecnológica y digital que dignifica el trabajo de los comerciantes de Francisco Z. Mena, cerrando la brecha digital desde las aulas del bachillerato tecnológico hacia el corazón productivo de la comunidad, demostrando que la ciencia y la tecnología son bienes sociales al servicio del desarrollo humano integral.

3. Yo Hago: Demostración y Protocolo Guiado por el Docente

Fases del Proyecto Formativo:

Fase 1: Diagnóstico técnico, levantamiento de requerimientos y diseño arquitectónico: Paso 1: Realizar una visita de campo o entrevista simulada con un comerciante o tallerista local de Francisco Z. Mena para identificar sus necesidades específicas de control de inventarios y ventas. Anotar los requerimientos funcionales (qué debe hacer el software) y no funcionales (rendimiento, seguridad, compatibilidad).

Paso 2: Traducir los requerimientos recopilados en un documento estructurado de especificación de requerimientos de software (DERS), estableciendo los módulos principales: módulo de usuarios (administrador y cajero), módulo de inventarios (altas, bajas, modificaciones, consultas y alertas de stock), módulo de ventas (punto de venta y cálculo de cambio) y módulo de reportes (ventas diarias y mensuales).

Paso 3: Diseñar el modelo de datos relacional del sistema. Crear un Diagrama de Entidad-Relación (DER) identificando las tablas esenciales: 'Usuarios', 'Productos', 'Categorías', 'Ventas' y 'Detalle_Venta'. Definir correctamente las llaves primarias (PK) y llaves foráneas (FK), asegurando la integridad referencial de los datos. <!--workbook:drawing:height=160-->

Paso 4: Elaborar los bocetos en papel o digital (Wireframes) de las pantallas principales del software, priorizando la simplicidad visual y la accesibilidad para usuarios con baja alfabetización digital. Las fuentes deben ser legibles y los botones de acción (Guardar, Cancelar, Cobrar) deben estar claramente diferenciados por colores.

Paso 5: Seleccionar el entorno de desarrollo integrado (IDE) y los lenguajes de programación a utilizar (por ejemplo, Python con Tkinter y SQLite, o JavaScript con Node.js y HTML/CSS). Verificar la instalación de librerías y dependencias necesarias para la ejecución local del proyecto en equipos con recursos limitados.

Advertencia técnica: Asegúrese de que el diseño de la base de datos evite la redundancia innecesaria y aplique al menos hasta la tercera forma normal (3FN) para prevenir inconsistencias futuras en el almacenamiento de precios y existencias.

Fase 2: Configuración del entorno, creación de la base de datos y programación del núcleo backend: Paso 1: Inicializar el proyecto en el control de versiones local (Git) y estructurar los directorios del proyecto de manera limpia: carpetas separadas para 'database', 'models', 'views', 'controllers' y 'assets'.

Paso 2: Escribir el script en lenguaje SQL para la creación automatizada de la base de datos relacional. Incluir restricciones como 'NOT NULL', 'UNIQUE' y 'CHECK' para evitar que se registren precios negativos o cantidades en inventario menores a cero.

Paso 3: Programar el módulo de conexión a la base de datos en el lenguaje seleccionado, implementando bloques de manejo de excepciones ('try-catch' o 'try-except') para capturar errores de conectividad o fallas en la ruta del archivo de base de datos.

Paso 4: Desarrollar la lógica de negocio (Backend) implementando las operaciones CRUD (Create, Read, Update, Delete) para la gestión del catálogo de productos. Cada función debe incluir validaciones estrictas de tipos de datos (por ejemplo, verificar que el precio ingresado sea estrictamente un valor numérico flotante).

Paso 5: Escribir y ejecutar pruebas unitarias en consola para verificar que la inserción, consulta, actualización y eliminación de registros en la base de datos se realicen de manera correcta y sin corromper la integridad de la información. <!--workbook:lines:rows=8-->

Advertencia técnica: Es fundamental sanitizar todas las consultas SQL mediante consultas parametrizadas para evitar ataques de inyección SQL ('SQL Injection'), incluso si la aplicación operará de manera local en una sola computadora.

Fase 3: Desarrollo de la interfaz gráfica de usuario (GUI) y módulo de punto de venta (POS): Paso 1: Construir las ventanas de la interfaz gráfica de usuario (GUI) basándose en los wireframes diseñados en la Fase 1. Asegurar una navegación fluida mediante pestañas, menús desplegables o botones de navegación intuitivos.

Paso 2: Programar el módulo de Punto de Venta (POS). Este módulo debe permitir al usuario buscar un producto por su código o nombre, agregarlo a un carrito de compras temporal, calcular de manera automática el subtotal, el impuesto aplicable (si lo hubiera) y el total general.

Paso 3: Vincular el módulo POS con la tabla de inventarios de la base de datos, de modo que cada vez que se concrete una venta, el sistema descuente de forma automática y exacta la cantidad vendida de las existencias actuales del producto.

Paso 4: Implementar un algoritmo de alerta visual (por ejemplo, coloreando en rojo las filas de la tabla de productos cuyas existencias sean menores o iguales al stock mínimo configurado por el comerciante).

Paso 5: Desarrollar el módulo de reportes que permita consultar las ventas realizadas durante la jornada diaria y calcular el total de ingresos obtenidos, mostrando los datos en una tabla clara o en un formato de texto estructurado listo para impresión. <!--workbook:lines:rows=8-->

Advertencia técnica: Evite bloqueos en la interfaz gráfica principal ('Freezing UI') al realizar consultas pesadas a la base de datos; si es necesario, implemente ejecución de procesos en segundo plano o optimice las consultas mediante índices en las columnas de búsqueda frecuente.

Fase 4: Depuración integral, pruebas de campo, empaquetado y entrega final comunitaria: Paso 1: Realizar una auditoría completa del código fuente para eliminar variables sin usar, código redundante, comentarios obsoletos y advertencias de compilación o interpretación.

Paso 2: Ejecutar pruebas de caja negra simulando situaciones de uso real y extremo (apagar el equipo abruptamente durante una venta, ingresar caracteres especiales en campos numéricos, saturar el inventario con cientos de registros) para evaluar la estabilidad y tolerancia a fallos del software.

Paso 3: Empaquetar la aplicación y sus dependencias en un archivo ejecutable portable o un instalador sencillo utilizando herramientas como PyInstaller (para Python) o empaquetadores equivalentes, asegurando que pueda ejecutarse en una computadora con Windows sin necesidad de instalar entornos de desarrollo complejos.

Paso 4: Redactar un Manual de Usuario ilustrado, redactado con un lenguaje sencillo, accesible y respetuoso, que explique paso a paso cómo abrir el programa, registrar un producto, realizar una venta y consultar el reporte diario.

Paso 5: Realizar una prueba de campo final entregando el artefacto al comerciante o tallerista local asignado en Francisco Z. Mena. Registrar sus comentarios, corregir cualquier detalle menor de usabilidad y documentar la entrega mediante fotografías y la firma de la bitácora de campo.

<!--workbook:lines:rows=8-->

Advertencia técnica: Compruebe rigurosamente que el ejecutable funcione en una computadora totalmente limpia (sin el entorno de desarrollo instalado) para garantizar que todas las librerías, iconos y archivos de bases de datos iniciales están correctamente empaquetados y referenciados por rutas relativas y no absolutas.

4. Nosotros Hacemos: Práctica Guiada en Equipo

Desarrollo colaborativo en equipos de trabajo con seguimiento docente en el aula y taller.

[Actividad] Diagrama de Arquitectura y Esquema de Conexión del Artefacto

Instrucción: Dibuja el diagrama de bloques, esquema funcional o circuito de tu solución:

[Espacio de dibujo técnico, diagramación o boceto a mano]

5. Tú Haces: Reto Autónomo de Aplicación

Desafío Autónomo ('Tú Haces'): Una vez concluido el ensamblaje y empaquetado del Sistema Integral de Control de Inventario y Ventas (SICIV-FZM), cada equipo de estudiantes deberá trasladarse de manera autogestiva al comercio, taller o miscelánea asignada en su comunidad de Francisco Z. Mena para realizar la instalación en el equipo de cómputo real del negocio. El estudiante deberá configurar el sistema introduciendo un inventario inicial real de al menos 20 productos proporcionados por el comerciante, incluyendo precios de compra, precios de venta y existencias actuales. Posteriormente, el estudiante capacitará durante un tiempo mínimo de 20 minutos al dueño o empleado del establecimiento, guiándolo en la realización de al menos 5 ventas simuladas y la consulta del reporte de corte diario. Durante este proceso, el estudiante deberá observar las dificultades de interacción que presente el usuario, tomar notas precisas en su bitácora de campo y realizar un ajuste menor de código de última hora si se detecta un fallo de usabilidad. Finalmente, recabará la firma de aceptación en la hoja de validación técnica por parte del comerciante y redactará una memoria técnica individual de dos cuartillas reflexionando sobre el impacto real de su software en la mitigación de la brecha digital local.

[Actividad] Lista de verificación de criterios de aceptación

Instrucción: Verifica junto con tu docente el cumplimiento de cada criterio antes de la entrega:

- [] Criterio 1: El software ejecutable inicia correctamente en menos de 5 segundos en un equipo de cómputo estándar de gama baja sin requerir instalación de software adicional.
- [] Criterio 2: Las operaciones CRUD en el módulo de inventarios operan sin errores, actualizando de forma inmediata y persistente la base de datos relacional.
- [] Criterio 3: El módulo de punto de venta (POS) descuenta con precisión matemática exacta el inventario tras cada transacción registrada y emite el cálculo correcto del cambio.

- [] Criterio 4: El sistema activa alertas visuales automáticas cuando un producto alcanza o rebasa el límite mínimo de stock establecido por el usuario.
- [] Criterio 5: Se incluye un manual de usuario impreso o digital claro, ilustrado y comprensible para personas sin formación técnica previa.

[Actividad] Bitácora de campo y lecciones aprendidas

Instrucción: Anota los principales retos técnicos enfrentados y cómo los resolvieron en equipo:

7. Punto de Control Formativo (Metacognición)

Pregunta formativa: ¿Qué impacto esperado tiene tu artefacto en la comunidad o en tu perfil profesional?

- Evalúa la viabilidad y durabilidad de tu propuesta.

Criterios de Verificación:

- [] El artefacto cumple los criterios de aceptación acordados.

MISIÓN 4: DEMOSTRACIÓN INTEGRAL Y EVALUACIÓN FORMATIVA — CONTROL DE CALIDAD, PRUEBA

Sesiones asignadas: 16, 17, 18 | Enfoque: Demostración de aprendizajes, coevaluación y rúbrica NEM

1. Enganche y Desafío Situado en la Comunidad

La evaluación formativa en el modelo NEM no es una prueba memorística; es la oportunidad en que tú y tu equipo demuestran ante la comunidad escolar cómo los conocimientos construidos en Programación responden a la problemática de Desarrollo de software y automatización digital comunitaria para comercios y talleres locales de Francisco Z. Mena, mitigando la brecha digital y optimizando el inventario comunitario..

Pregunta Detonadora: ¿De qué manera tus evidencias demuestran que alcanzaste las metas de aprendizaje y qué impacto real tienen tus soluciones?

2. Concepto Cero: Analogía Intuitiva y Fundamento

Analogía Física Cotidiana: La evaluación es como el control de calidad final en una línea de producción: no busca descartar piezas, sino asegurar que cada una cumpla con la resistencia y confiabilidad requerida para su propósito real.

La evaluación formativa combina tres miradas indispensables: la heteroevaluación docente guiada por una rúbrica analítica, la coevaluación técnica entre compañeros mediante lista de cotejo, y la autoevaluación reflexiva sobre tu propio progreso cognitivo.

3. Yo Hago: Demostración y Protocolo Guiado por el Docente

Rúbrica Analítica Oficial NEM:

Criterio: Rigor metodológico y aplicación procedimental en el desarrollo de software (Ponderación: 25%)

- ****Sobresaliente / Excelente (10 pts):**** El estudiante demuestra un dominio absoluto en la aplicación de las fases del ciclo de vida del desarrollo de software, estructurando algoritmos limpios y aplicando paradigmas de programación eficientes adaptados a la problemática del comercio local en Francisco Z. Mena. Se observa una planeación exhaustiva, el uso correcto de diagramas de flujo y una traducción impecable a código fuente validado sin errores de compilación ni de lógica.
- ****Notable / Bueno (8 pts):**** El estudiante aplica los procedimientos metodológicos establecidos con precisión y solvencia técnica, mostrando un entendimiento claro de la lógica de programación. Aunque la estructura general del software es funcional y responde al contexto comunitario, presenta mínimas omisiones en la documentación preliminar o requiere ligeras correcciones en la optimización de ciclos y condicionales complejos.
- ****Suficiente / Básico (6 pts):**** El estudiante cumple de manera elemental con los procedimientos requeridos para el desarrollo de la aplicación, reflejando un esfuerzo por resolver la tarea asignada. Sin embargo, se aprecian deficiencias notables en la estructura modular del código, uso limitado de buenas prácticas de sintaxis y una dependencia constante de la guía docente para superar bloqueos lógicos básicos durante la codificación.
- ****Insuficiente / Requiere Apoyo (4 pts):**** El estudiante presenta graves inconsistencias metodológicas y conceptuales que impiden el funcionamiento correcto del software. No sigue un orden lógico en el diseño algorítmico, el código fuente está incompleto o carece de estructura, y no logra vincular la solución técnica con las necesidades de digitalización planteadas en el proyecto comunitario del municipio de Francisco Z. Mena.

Criterio: Funcionalidad, validación de datos y robustez del código (Ponderación: 25%)

- ****Sobresaliente / Excelente (10 pts):**** El programa informático desarrollado opera con máxima eficiencia, integrando rutinas robustas de validación de entradas de datos que evitan caídas del sistema ante valores inesperados. La gestión de inventarios para los talleres locales funciona en tiempo real con precisión milimétrica, demostrando pruebas exhaustivas de caja negra y blanca que garantizan la estabilidad absoluta del artefacto digital bajo cualquier escenario operativo.
- ****Notable / Bueno (8 pts):**** El software cumple satisfactoriamente con los requerimientos funcionales solicitados, procesando la información de manera correcta y estableciendo mecanismos estándar de validación de datos. Las pruebas realizadas cubren los casos de uso principales para los comercios locales, aunque pueden presentarse fallos menores en situaciones límite o entradas de usuario no contempladas inicialmente en el diseño.
- ****Suficiente / Básico (6 pts):**** El artefacto digital muestra una funcionalidad limitada a los procesos más simples y directos, careciendo de mecanismos avanzados de control de excepciones y validación robusta de datos. El sistema es propenso a errores ante ingresos de información imprevista y la gestión del inventario presenta intermitencias que afectan la confiabilidad general de la herramienta propuesta para la comunidad.
- ****Insuficiente / Requiere Apoyo (4 pts):**** El software carece de funcionalidad operativa básica; presenta errores críticos de ejecución, desbordamientos de memoria o bucles infinitos que imposibilitan su uso práctico. No cuenta con validación de datos, provocando que el programa colapse frecuentemente y demostrando una ausencia total de pruebas de calidad antes de su presentación ante los comercios beneficiados.

Criterio: Impacto socio-comunitario y vinculación con el PAEC en Francisco Z. Mena (Ponderación: 25%)

- ****Sobresaliente / Excelente (10 pts):**** El proyecto de software se vincula de manera excepcional y transformadora con la problemática de la brecha digital y la optimización de inventarios en los comercios y talleres de Francisco Z. Mena. El estudiante demuestra empatía social y compromiso ciudadano, diseñando interfaces sumamente intuitivas, accesibles y adaptadas a usuarios con nula experiencia tecnológica previa, logrando un impacto medible y altamente positivo en la economía local.

- ****Notable / Bueno (8 pts):**** La propuesta digital aborda de manera clara y directa las necesidades planteadas por el PAEC comunitaria, ofreciendo una solución útil para los pequeños negocios del municipio. La interfaz de usuario es comprensible y funcional, facilitando la adopción tecnológica, aunque se beneficiaría de un estudio más profundo sobre las dinámicas comerciales específicas de la región para maximizar su arraigo y utilidad.
- ****Suficiente / Básico (6 pts):**** El proyecto muestra una relación tangencial con la problemática comunitaria de Francisco Z. Mena, cumpliendo con el requisito formal pero sin lograr una verdadera empatía con las necesidades reales de los comerciantes locales. La interfaz resulta compleja o poco amigable para usuarios no especializados, lo que limita su potencial de aplicación directa en el entorno socioeconómico del municipio.
- ****Insuficiente / Requiere Apoyo (4 pts):**** El software desarrollado está completamente desconectado de la realidad comunitaria y de los objetivos del PAEC. No considera las limitaciones tecnológicas, de conectividad o de capacitación de los talleres locales de Francisco Z. Mena, resultando en un producto irrelevante, descontextualizado y sin ningún valor práctico para mitigar la brecha digital en la región.

Criterio: Documentación técnica, estilo de codificación y trabajo colaborativo (Ponderación: 25%)

- ****Sobresaliente / Excelente (10 pts):**** El código fuente presenta un estándar sobresaliente de limpieza, con identificación perfecta, uso riguroso de nomenclatura significativa para variables y funciones, y comentarios explicativos claros que facilitan su mantenimiento futuro. La documentación técnica y el manual de usuario están redactados con rigor profesional, reflejando una excelente organización y distribución de roles dentro del equipo de trabajo durante todo el proceso de desarrollo.
- ****Notable / Bueno (8 pts):**** El código fuente mantiene un estilo ordenado, legible y consistente, siguiendo las convenciones de nomenclatura recomendadas para el lenguaje de programación utilizado. La documentación técnica describe adecuadamente los componentes principales del sistema y el manual de usuario cumple su propósito, respaldado por una colaboración efectiva y responsable entre los integrantes del equipo de trabajo.
- ****Suficiente / Básico (6 pts):**** El código fuente es funcional pero desorganizado, con problemas frecuentes de identificación, nombres de variables poco descriptivos y escasa o nula documentación interna. La documentación técnica es superficial, incompleta y el trabajo colaborativo mostró desequilibrios notables en la participación y en la comunicación asertiva entre los miembros del equipo de desarrollo.
- ****Insuficiente / Requiere Apoyo (4 pts):**** El código fuente es caótico, carece de estructura lógica visible, utiliza nombres de variables arbitrarios y no incluye ningún tipo de comentario explicativo. La documentación técnica brilla por su ausencia y el proceso de trabajo colaborativo fue deficiente, evidenciando conflictos internos, falta de planeación y un incumplimiento sistemático de las metas parciales asignadas.

4. Nosotros Hacemos: Práctica Guiada en Equipo

Sesión de coevaluación técnica: los equipos contrastan sus entregables utilizando la lista de cotejo y registran observaciones constructivas.

[Actividad] Lista de Cotejo Técnica del Artefacto (Coevaluación entre Pares)

Instrucción: Intercambia tu proyecto con un compañero y verifica objetivamente el cumplimiento de cada indicador:

- [] [Seguridad] El estudiante verifica y aplica protocolos de seguridad eléctrica y ergonómica antes de iniciar la manipulación del equipo de cómputo.
- [] [Seguridad] El entorno de desarrollo integrado (IDE) está correctamente instalado, configurado y actualizado con las librerías necesarias para el proyecto.
- [] [Metodología] El algoritmo principal cuenta con un diagrama de flujo previo que plasma de forma lógica la solución al problema de inventario.
- [] [Funcionalidad] El código fuente implementa correctamente estructuras de control secuenciales, selectivas y repetitivas según los requerimientos.
- [] [Metodología] Las variables y funciones utilizan nomenclatura estándar y descriptiva que facilita la lectura y comprensión del código.
- [] [Funcionalidad] Se han integrado rutinas de validación de entradas de datos para prevenir fallos por inserción de caracteres alfanuméricos incorrectos.
- [] [Funcionalidad] El sistema realiza el registro, actualización y consulta de inventarios en tiempo real sin pérdida de información.
- [] [Funcionalidad] Se incluye un módulo de control de excepciones (try-catch o equivalente) para evitar cierres abruptos del programa.
- [] [Presentación] El diseño de la interfaz de usuario es intuitivo, accesible y adecuado para personas con baja alfabetización digital en talleres locales.
- [] [Metodología] El programa incluye comentarios internos claros y estructurados que explican la lógica de los bloques de código más complejos.
- [] [Seguridad] Se realizó un respaldo periódico del código fuente utilizando un sistema de control de versiones (ej. Git/GitHub) durante el desarrollo.
- [] [Presentación] El proyecto incluye una guía de usuario redactada en lenguaje sencillo y accesible para los comerciantes de Francisco Z. Mena.
- [] [Funcionalidad] Se validó el funcionamiento del software mediante pruebas de caja negra simulando escenarios reales de tiendas abarroterías locales.
- [] [Impacto PAEC] El software se adapta a las limitaciones de hardware comunes en los pequeños negocios de la región (equipos de cómputo básicos o seminuevos).
- [] [Impacto PAEC] La solución propuesta mitiga directamente la brecha digital identificada en el diagnóstico comunitario del PAEC.
- [] [Presentación] La presentación final del proyecto demuestra claridad expositiva, dominio técnico y una sólida vinculación social con la comunidad.

5. Tú Haces: Reto Autónomo de Aplicación

Autoevaluación individual y resolución de los reactivos de juicio crítico en el cuaderno de trabajo.

[Actividad] Bitácora de Autoevaluación Metacognitiva

Instrucción: Responde reflexivamente a las preguntas de consolidación del bloque:

7. Punto de Control Formativo (Metacognición)

Pregunta formativa: ¿Qué evidencias concretas avalan el nivel de desempeño que asignaste a tu proyecto?

- ¿De qué manera el desarrollo de este bloque de Programación transformó tu percepción sobre la utilidad social de la tecnología, y cómo puedes aplicar estos conocimientos para mitigar la brecha digital en tu propia comunidad dentro del municipio de Francisco Z. Mena?
- ¿Cuál fue el obstáculo lógico o técnico más complejo al que te enfrentaste durante la codificación del software de inventario, y qué estrategia específica implementaste para superarlo con éxito?
- ¿Cómo influyó el trabajo en equipo y la comunicación asertiva con tus compañeros en la calidad final del artefacto digital entregado para los talleres y comercios locales?
- ¿Qué aspectos éticos y de seguridad consideras indispensables al manejar datos sensibles de inventarios y transacciones comerciales de pequeños negocios locales, y cómo los integraste en tu proyecto?
- Si pudieras mejorar o escalar tu aplicación informática actual en el futuro, ¿qué nuevas funcionalidades le añadirías para maximizar su impacto positivo en la economía de la región?
- ¿Qué errores comunes cometiste al inicio del diseño algorítmico y qué aprendizajes te dejaron para futuros proyectos de desarrollo de software?

Criterios de Verificación:

- ☐ Contrasté mi producto contra todos los niveles de la rúbrica analítica.
- ☐ Completé la lista de cotejo con imparcialidad técnica.
- ☐ Redacté mis compromisos de mejora para el siguiente periodo.